天体力学 × AI · 2026

从下一个 token 到下一个定理



刘博文

刘博文 Bowen Liu

研究主线：N 体周期解、椭圆相对平衡与线性稳定性。

2013–2018 **南开大学** · 陈省身数学研究所 · 数学博士

2018–2020 **奥格斯堡大学** · 博士后

2020–2022 **上海交通大学** · 吴文俊助理教授

2022–2024 **华为** · 主任工程师

2024–至今 **某美国电商公司** · 应用研究员

天体力学研究

N 体周期解的存在性与稳定性

受限四体：椭圆相对平衡

平面四体：椭圆菱形解的不稳定性

广义 Lennard–Jones 系统的同形解

辛指标理论 · 辛几何 · 变分方法

全景：从 token 到定理

点击章节卡片跳转；点击小标签直达关键页。

00 · 开场与 TOKEN

Token

十个整数 · 词表 · 计费

p. 4–9

简介 · 10 个整数 · 词表

01 · 模型

一个函数

概率分布

p. 10

概率分布

02 · 训练与问题求解

从奖励中学习

RL 循环 · 数学与代码 · GRPO

p. 11–23

RL 循环 · GRPO 公式 · 无状态

03 · 系统与工程

把模型放进系统

上下文 · Harness · Skill · 长时运行

p. 24–31

Harness · 系统图 · Skill · 长时运行

04 · AI × 数学

从工具到系统

加速 · 机制 · 定理爆破系统

p. 32–43

DAG/MCTS · 722 篇 · NS · 爆破系统

05 · 结论

从 token 到定理

概率 · 验证 · 方向

p. 44

结尾

模型看到的，是 10 个整数

EN · 一句话

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 个字符 → **10** 个 token → 10 个整数 · _ 表示前导空格

中文 · 《天体力学基础》内容简介

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 个字符 → **10** 个 token → 10 个整数 · 系统 / 介绍 各占一个 token

01

这串整数是怎么来的?

02

它怎么预测下一个?

03

我们怎么用它干活?

切分是查表的结果

三体问题 is a classic problem in celestial mechanics.

三	体	问题	_is	_a	_classic	_problem	_in	_celestial	_mechanics	.
3236	7074	28917	382	261	13686	4792	306	134437	52347	13

_ 表示前导空格 · Celestial → Cel + estial

≈5
字符 / 英文单词
含空格，常见值

≈4
字符 / token
英文经验值

≈1.3
token / 英文单词
≈ 5 ÷ 4

5–20 万
常见词表大小
50k – 200k

为什么 字母级：信息少，序列长 4–5 倍 → 词级：词表爆炸（百万级） → **子词：折中**

> 子词不是语义单位，是**序列长度**与**词表大小**之间的工程折中。

词表：一张固定的映射表

ID	片段	
867	天	
7074	体	
8590	学	
11343	力	
20121	书	
31936	系统	
78580	基础	← 本次查询
199999	<|endoftext|>	特殊

只展示 8 行　　共 200,019 行

词表大小 = 行数

200,019

o200k · GPT-4o 代

- **0**
- **50k** GPT-2
- **100k** GPT-4 代
- **128k** Llama 3
- **200k** GPT-4o 代

200,019 × 4,096 ≈ **8.2 亿参数**

> ID 是**行号**；换模型，就是换一张表。

Token 是模型的单位，也是账单的单位



≈ 0.3 token

1 个英文字符

≈ 0.6 token

1 个中文字符

> Token 既是模型的最小单位，也是账单的最小单位。

一次调用，钱花在哪

价格[2]	百万tokens输入（缓存命中）	空闲时段	0.02元
		高峰时段	0.04元
	百万tokens输入（缓存未命中）	空闲时段	1元
		高峰时段	2元
	百万tokens输出	空闲时段	4元
		高峰时段	8元

一次调用 · 空闲时段

输入 100k + 输出 2k

¥0.01 缓存命中

¥0.11 缓存未命中

> 同一段上下文，命中缓存与不命中，差一个**数量级**。

文字先变成 ID，再变成向量

片段

问题

切分后的单位

→

ID

28917

词表第 28917 行

→

向量

0.12 −0.44 0.08 …

4096 维 · 训练得到

> ID 是**地址**，向量是**内容**。

模型的工作：输入 → 概率分布

输入 token

从 | 下一个 | token | 到 | 下一个

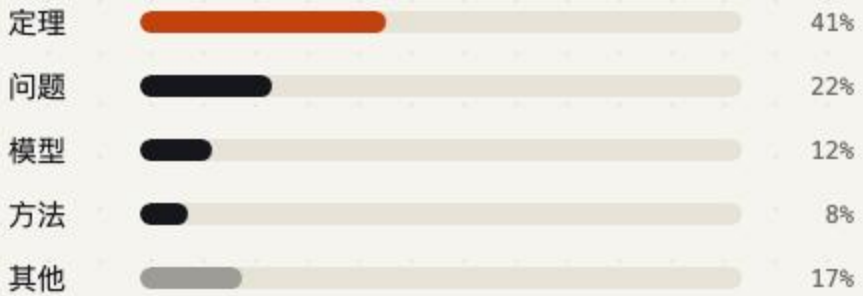
每个 token 已经是一个向量

模型

Transformer

N 层 · 自注意力

softmax(z)



$$z = f_\theta\left(x_{1:t}\right) \in \mathbb{R}^{|V|} \qquad p_\theta\left(y \mid x_{1:t}\right) = \frac{\exp\left(z_y\right)}{\sum_{v \in V} \exp\left(z_v\right)}$$

$|V| = 200{,}019$

$\Sigma_y\, p_\theta(y \mid x_{1:t}) = 1$

> 它不查数据库，也不“理解”；它给词表里每个 token **打分**，再把分数变成**概率**。

训练：预训练与后训练

阶段 01 · PRETRAINING

预训练

大规模文本、代码等语料
学习语言、知识与模式

→

阶段 02 · POST-TRAINING

后训练

SFT · 监督微调

从示范学习

学习回答方式与任务行为。

RL · 本节重点

通过奖励学习

尝试、评价、改进策略。

> 重点讲 **RL**：奖励如何推动**问题求解能力**提升。

RL：行动、结果与反馈

在环境里行动，根据结果反馈，逐步改进动作选择。

观察环境 → 采取动作 · Action → 得到结果 → 反馈 / 奖励 → 改进策略 ↺

环境 · ENVIRONMENT

产生结果

动作改变环境，返回新的观察。

反馈 · FEEDBACK

评价结果

用奖励表达进展与成败。

策略 · POLICY

改进动作选择

根据经验调整，追求长期回报。

> RL 根据**结果反馈**，学习**怎样选择动作**。

游戏，让强化学习的能力被看见

2015

Atari · DQN

像素 → 动作

奖励：游戏分数

2016

AlphaGo

自我对弈 + 搜索

奖励：赢棋

2019

OpenAI Five

Dota 2 团队协作

奖励：比赛结果

2019

AlphaStar

StarCraft II 策略

奖励：比赛结果

- 规则明确
- 结果可衡量
- 可反复尝试

> 游戏引擎同时提供**环境与反馈**。

一个分数，怎样教会一种策略?

打砖块：移动挡板，接住球，清除砖块。

尝试

不同动作

探索时机与方向。

观察

结果得分

漏球，或清除更多砖块。

学习

更好的策略

学会具有延迟收益的动作。

反复尝试 + 结果反馈 → 学习取得更高回报的策略。

有用的环境，让智能体**尝试、评价、再尝试**。

GRPO 的目标函数

DeepSeekMath：组内相对优势，加上裁剪与 KL 约束。

$$J_{\mathrm{GRPO}}\left(\theta\right)=E_{q\sim P(Q),\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\mathrm{old}}}}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}L_{i,t}\right]$$

逐 TOKEN 项

$$L_{i,t}=\min\left[\rho_{i,t}\left(\theta\right)\hat{A}_{i,t},\mathrm{clip}\left(\rho_{i,t}\left(\theta\right),1-\varepsilon,1+\varepsilon\right)\hat{A}_{i,t}\right]-\beta D_{\mathrm{KL}}\left[\pi_{\theta}\middle\|\pi_{\mathrm{ref}}\right]$$

概率比

$$\rho_{i,t}\left(\theta\right)=\frac{\pi_{\theta}\left(o_{i,t}\mid q,o_{i,<t}\right)}{\pi_{\theta_{\mathrm{old}}}\left(o_{i,t}\mid q,o_{i,<t}\right)}$$

组内优势

$$\hat{A}_{i,t}=\frac{r_i-\mathrm{mean}\left(r_1,\ \ldots\ ,r_G\right)}{\mathrm{std}\left(r_1,\ \ldots\ ,r_G\right)}$$

> 提高高优势回答的概率，同时用 **clip 与 KL** 限制更新幅度。

GRPO：同一道题，比较一组回答

不用 critic，用组内相对表现构造优势。

同一问题 → 采样一组回答 → 逐个评分 → 组内相对优势 ↺

同一问题 · G = 4 个回答

奖励 / 组内优势 / 更新方向

A · r = 0.20	−1.5 ↓
B · r = 0.90	**+1.1 ↑**
C · r = 0.50	−0.4 ↓
D · r = 0.80	**+0.7 ↑**

组内竞争

同一题的回答互相比较，不跨题比较绝对分数。

优势决定更新方向

- $\hat{A} > 0$ 提高这个回答的概率
- $\hat{A} < 0$ 降低这个回答的概率
- $\hat{A} \approx 0$ 没有相对优势，梯度贡献很小

不需要 CRITIC

组均值就是基线，省掉单独的价值模型。

同一道题里，让**更好的回答胜出**。

数学与代码，共享一类反馈优势

01 · 外部环境封闭

公理系统 · 运行环境

数学靠定义、公理与推理规则；代码靠语言规范与运行时。系统自足，不需要外部信息。

02 · 错误可追溯

定位到具体步骤

证明逐步检查，找到失效步骤；测试、异常栈与行号定位代码错误。

03 · 数据干净

可解析、可筛选

形式化对象与代码结构清晰，便于批量生成、过滤与验证。

两条路线：学习更好的策略 + 用算力生成、验证更多候选。

封闭环境 + 可定位错误 + 干净数据 → **反馈便宜，可大规模试错**。

代码与数学，训练共享的推理技能

编程训练

- 追踪变量与状态
- 遵守类型与约束
- 分解问题，定位错误

数学训练

- 明确写出假设
- 保持符号与逻辑一致
- 建立并检查多步论证

已有迁移证据： 数学 RL → 代码表现；代码训练 → 状态追踪。

共享技能支持**能力迁移**，数据配比影响实际收益。

编程，为 Agent 提供了行动场所

读文件 → 修改 → 跑测试 → 观察与修订 ↺

工作目录

文件与规范定义任务。

真实工具

修改可以立即执行。

结果反馈

失败指导下一次修改。

> 代码在同一工作目录连接**生成、行动与反馈**。

四类任务，不同的反馈条件

编程、数学、产品设计、交易：目标、环境与反馈怎样配合?

01 · 编程

可执行的检查

- 目标：满足指定功能与规范
- 反馈：编译、测试、报错
- 试验：本地反复运行与修订

02 · 数学

明确的假设与目标

- 目标：在给定假设下求解或证明
- 反馈：答案检查、证明检查、反例
- 试验：反复生成与验证候选

03 · 产品设计

长周期的现实反馈

- 目标：用户价值与差异化
- 反馈：更慢、多元，需现实证据
- 判断：独特路线可能来自非共识

04 · 交易

清晰奖励，变化的环境

- 目标：盈亏、风险与成本
- 环境：信息不完备，市场持续变化
- 验证：回测与未来收益有距离

> 可检查的目标，让更多算力能用于**系统搜索与候选筛选**。

正确是底线，优秀是追求

	代码	数学
可检查结构	语法、类型、测试、规范	**定义、假设、证明步骤**
正确性	满足指定行为	**命题从假设推出**
优秀程度	清晰、可维护、高效	**优雅、一般、有用**

> 两个领域都追求**好作品**，超越一次通过或失败。

总结：环境、反馈与参数更新

良好定义的问题，让模型可以在环境中反复预测、接受检查并迭代。

问题与环境 → next-token 预测 → 检查与追溯 → 定义 loss → 反向传播 ↻

01 · 环境

规则与目标明确

模型可以执行、尝试并产生输出。

02 · 反馈

评判与追溯

检查结果，定位出错步骤。

03 · 更新

loss 驱动迭代

反向传播，更新参数，提升能力。

环境提供反馈，**loss 把它变成参数更新**。

模型只会预测下一个 token

记忆、计算、工具与计划，都在这个动作外面。

前文 token → 模型 → 下一个 token 的概率 → **采样**

唯一动作：给下一个 token 打分，采样一个。

01 · 不记住上一轮

对话历史由客户端每次重发

02 · 不做真正计算

算术是记忆，不是现场计算

03 · 不直接行动

工具由外部执行

04 · 不维护目标

计划也是生成的文本

> 模型只给下一个 token；**其余能力都从系统里来。**

训练之后：四个短板，三类系统能力

模型会预测；状态、计算与可靠性还在模型之外。

01 · 记忆与上下文

不保存状态，窗口有限

- 参数推理时不更新；知识在权重里
- “对话”是客户端重发历史
- 4k → 1M，但长文仍会“大海捞针”

02 · 计算是记忆

记忆与模式补全

- 算术来自训练分布，不是执行算法
- 像九九表：调用记忆，不是算法
- 长程任务会累积误差

03 · 概率输出

同一输入，也可能不同

- 输出是分布采样，不是确定函数
- 错误可能自信而合理
- 长时运行需要停止与回滚

> 四个短板，三类系统能力：**状态、计算、可靠性**。

Context window 的膨胀



贵

按 token 计费；长上下文预填更慢

慢

prefill 与注意力成本上升

稀释

中间信息被忽略；信号被噪声淹没

> 信息密度比窗口长度更重要。

Context 与 Memory：有限窗口，怎么放

窗口是预算，不是仓库：先放对，再决定留什么。

01 · 窗口有限

4k → 1M，仍会丢

大海捞针：长文中间的信息最容易被稀释。



容量 ≠ 有效利用。

02 · 怎么放

工作台，不是仓库

- system · 规则
- 历史 · 前几轮
- 检索 · 相关文档
- 工具 · 返回结果
- 当前 · 这次的问题

太长→检索；太散→结构化；超窗→摘要或子 agent。

03 · 留什么

信号，不是日志

信号	噪声
任务 · 约束 · 决定 · 事实	闲聊 · 重复 · 过期 · 日志

写
只存决定与事实

取
按当前任务

压
摘要、合并

忘
过期即删

工作 · 情节 · 语义 · 程序

> 窗口容量 ≠ 有效利用；Context 决定**放什么**，Memory 决定**留什么**。

Tool calling：算不准，就交给程序

```
tool schema
{
  "name": "run_python",
  "args": {"code": "str"},
  "returns": "str",
  "limits": ["sandbox", "timeout"]
}

one call
call  run_python("print(2 * 4**1.5)")
obs   "16.0"

// T = 2 × 4**1.5 = 16
```

01 定义

名字、参数、返回、边界；工具描述也进入上下文。

02 调用

模型输出 tool_call；只提议，不执行。

03 执行

运行时校验参数，沙箱里真的跑；超时与权限由系统管。

04 观察

stdout / stderr 回到上下文；模型决定继续或停止。

> 工具输出仍是**不可信输入**：校验参数、限制权限、设置超时、保留日志。

> 模型提议，程序执行，**系统管边界**。

Harness：让概率输出变成可运行的系统

模型会提议；harness 负责让提议合法、可执行、可检查、可停止。

一个受控循环

提议 → 校验 → 执行 → 观察 → 验证 → 继续 / 停止

工程边界

模型只负责生成概率分布与调用提议；
外部环境的安全性与确定性，完全由 **Harness** 兜底。

> 不提升模型智力；让错误**可发现、可停止、可复现**。

01 工具注册与参数校验
输入是否合法、类型是否匹配；非法提议直接拦截

02 沙箱与权限
能做什么、不能做什么；只读/写分离，先限制再执行

03 超时与重试
失败不无限死循环；设置超时熔断与最大重试次数

04 结果验证
结果对不对；通过独立测试与断言检查，不轻信模型自夸

05 回滚与停止
坏了能否退回快照；达到步数或异常时强制刹车

06 日志与复现
记录每一次调用与返回，全流程轨迹可审计、可复盘

> 模型提供概率，harness 提供**边界**。

把模型放回系统里



01	**LLM**	概率生成器：输入 token，输出下一个 token 的分布
02	**Prompt**	这次的条件：system / user / few-shot
03	**Context**	这次能看到的全部 token；窗口有限
04	**Memory**	窗口外的状态：写入 / 检索 / 压缩 / 遗忘
05	**Agent loop**	观察 → 行动 → 观察；状态放到外部
06	**Harness**	工具、沙箱、权限、验证、日志

> Memory 管状态，Tool calling 管计算，Harness 管验证：三者合起来才是 **Agent 系统**。

Skill：把一套做法封装成可复用能力

不是更大的模型，而是按需加载的流程、脚本与参考资料。

```
skill package
skill/
  SKILL.md        when · steps · checks
  scripts/        deterministic ops
  references/     domain docs
  assets/         templates
```

提纲 → 生成 → 截图 → 审计 → 导出

01 触发
名称与描述：什么时候该用，什么时候不该用

02 步骤
SKILL.md：先做什么、后做什么、检查点在哪

03 工具
scripts/：把确定性操作固化，减少每次重写

04 资产
references/ 与 assets/：领域知识、模板、示例

Skill 按需加载、可版本化、可复用；仍受上下文预算与工具权限约束。

把一次性的提示，变成**可版本化的流程**。

长时间运行：把 loop 固化成 skill

loop 产生经验；skill 把经验变成下一轮可加载的流程。

01 · 跑起来

Loop

观察 → 行动 ↺

状态放到外部；可以持续数小时或数天。

02 · 提炼

轨迹 → 步骤

成功 失败 修复 检查点

去掉偶然，留下可重复的部分。

03 · 固化

Skill

```
SKILL.md
scripts/
references/
```

按需加载，下一轮直接用。

> loop 是运行时；skill 是编译后的经验；memory 是状态；harness 是边界。

> 长时间运行不只完成任务，也把做法变成**资产**。

加速：从基本应用到金牌

2023 · 之前

基本应用

- 解释概念、查文献、写代码
- 做简单计算，经常算错
- 复杂证明仍要人
- 角色：助手

→ 2024–26

2025–26 · 之后

研究级

- IMO 2025 金牌级 35/42
- Lean 形式证明可验
- OpenAI 2026：每项结果约 3 Pro 小时算力*
- 角色：候选生成器

2024.07 **28/42** 银牌 → 2025.07 **35/42** 金牌级　12 个月，+7 分

> 这是一条阶梯：**三个台阶**。

时间线：三个台阶

2023–24
工具：搜索 + 验证

2023.12
FunSearch
LLM + evaluator + 进化搜索；cap set 新构造

2024.01
AlphaGeometry
神经符号；IMO 几何 25/30

2024.07
AlphaProof
Lean + RL；IMO 2024 银牌 28/42（+AG2）

2024–25
推理：RL + 测试时计算

2024.09
OpenAI o1
RL 学思考；test-time compute

2025.01
DeepSeek-R1
RLVR；推理行为涌现；开放权重

2025.05
AlphaEvolve
进化编码 agent；4×4 复矩阵乘法 48 次

2025–26
规模：从 benchmark 到开放问题

2025.07
IMO 2025
金牌级 35/42；自然语言、无工具

2025
形式化金牌（声称）
Aristotle 等；Lean 可验

2026.05–10
OpenAI
单位距离反例；NS / 黎曼相关（声称）；首发 372 结果族

> 每一次跃迁，都是**生成器 + 验证器 + 搜索**。

他们怎么说的

> 公开材料覆盖了方法的一部分；完整的训练配方仍未公开。注意一种不对称：有些结果验证容易，证明很难。下面是论文、博客和赛后声明里的**公开表述**（节选 / 意译）。

时间	系统 / 工作	公开表述	谈法
2023.12	**FunSearch**	"pairing a pre-trained LLM ... with an automated evaluator"	搜索
2024.01	**AlphaGeometry**	"100 million synthetic theorems and proofs"	合成数据
2024.07	**AlphaProof**	autoformalization → Lean; RL + test-time RL	形式化
2024.09	**o1**	"spend more time thinking through problems"	推理
2025.01	**R1**	"directly apply RL ... rule-based rewards"	RLVR
2025.05	**AlphaEvolve**	"evolutionary coding agent"	进化搜索
2025.07	**IMO 2025**	"natural language ... no tools"	端到端
2026.09–10	**OpenAI**	NS 破裂声称；首发 372 结果族、722 篇手稿	开放问题

> 关键词在变，底层动作没变：**提议、验证、再搜索**。

共同的模式：生成 → 验证 → 搜索

生成

LLM

候选证明、程序、构造

→

验证

Lean / 测试 / 数值

给出对错、分数、反例

→

搜索 / 学习

best-of-N · 树 · 进化 · RL

把信号变成下一步策略

↺ 回到生成器：下一轮更强

没有验证器

只能靠人看；无法规模化

没有搜索

一次生成；好答案靠运气

两者都有

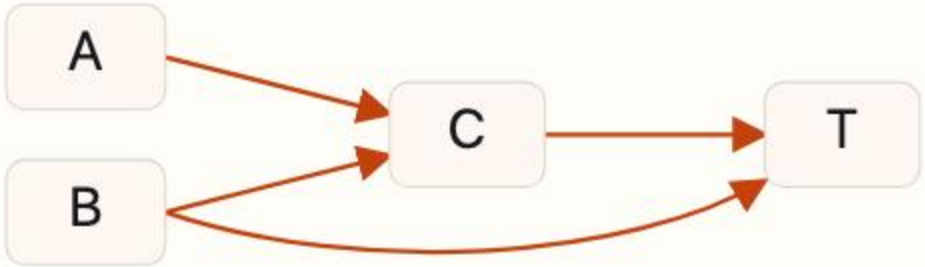
概率被筛成正确，再写回模型

> 模型提供概率，**验证器提供梯度**。

DAG 存依赖；MCTS 决定往哪里搜

DAG · DIRECTED ACYCLIC GRAPH

有向无环图：一种结构



节点：引理、子目标

有向边：前提 → 结论

无环：避免循环论证；共享节点可复用

MCTS · MONTE CARLO TREE SEARCH

蒙特卡洛树搜索：一种策略



节点：状态；分支：候选动作

选择 → 扩展 → 模拟 / 评估 → 回传

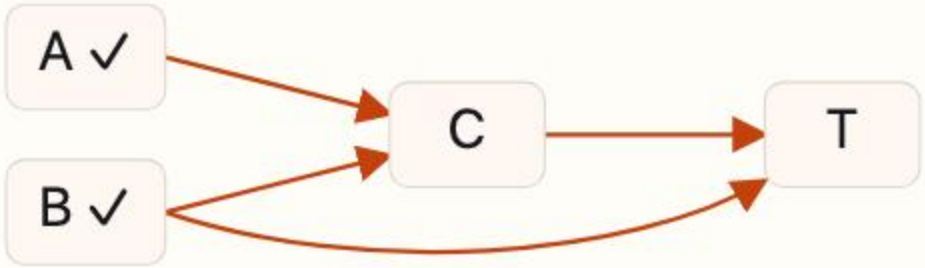
平衡探索与利用，分配搜索预算

DAG 回答“**依赖什么**”；MCTS 回答“**下一步试什么**”。

我的猜测：Agent + Lean 推动证明图演进

外部维护中间图 / 树：待证目标、候选路径、已证引理与验证反馈。

01 · 维护结构：目标、依赖与已证引理



节点存子目标；边记录依赖

已证引理固化，供其他节点复用

02 · 演进结构：提议、检查、更新



选节点 → Agent 尝试 → Lean 检查

通过则固化；失败则修正或换路

图存中间状态，Agent 扩展候选，Lean 把关证明。

我的判断：进展显著，技术路线仍在成熟

OpenAI 披露尝试约 4,000 个问题；发布结果覆盖其中一部分。

01 · 系统设计的瓶颈

如何组织搜索

目标拆解与节点选择

分支协作与中间状态共享

预算分配与验证反馈

万级并发的长时稳定

02 · 模型能力的瓶颈

能否找到关键一步

提出有效引理与新构造

跨越长程推理的难点

将思路写成严格证明

> Lean 守住形式逻辑；**系统与模型共同决定能走多远。**

OpenAI：批量生成研究级数学结果

从单题演示，到一整批可供审查与接续研究的手稿。

722	372	≈4,000
首发手稿	相关结果族	尝试的问题

公开了什么

论文与证明材料
部分结果的 Lean 形式化
10 份推理摘要与算力统计

我怎么读这组数字

结果族可含多篇手稿
约 3 Pro 小时：算力等价值
正确性、原创性、重要性待审

> 候选成果开始批量生成；**验证与理解成为关键工作**。

Navier–Stokes：一次长程多 agent 研究

OpenAI 报告：带光滑外力的三维不可压流体，可在有限时间失去光滑性。

$$\partial_t u + (u \cdot \nabla) u = -\nabla p + \nu \Delta u + f, \quad \nabla \cdot u = 0$$

结果范围 · 公司声称

正黏性 · 光滑初值与外力

全空间 $\mathbb{R}^3$ 与周期情形

有限时奇性 · 对应 Clay C / D

构造破裂情形；无外力问题另需讨论。

≈10^4	88 h	+17 h	130B
并发 agent	找到论证	形式化与验证	输出 token

分组探索 → 汇总中间结论

交叉分享 → 继续搜索 → Lean 检查

万级并发 → 调度、检查点、重试、停止自动化

形式证明检查逻辑；**原问题、形式陈述与论文的对应仍须审查**。

定理爆破系统：LLM 是 CPU

问题不只是一道数学题，而是一台机器的吞吐与稳定性。

工作负载	**定理 · 证明图**	解什么
CPU	**LLM**	提议下一步
调度	**Loop + Harness**	并发 · 重试 · 检查点 · 停止
验证	**Lean / 工具**	对错与计算
状态	**Context + Memory**	图 · 日志 · 预算

01 · 不只是数学

拆解、调度、状态、验证都是工程

02 · 吞吐

并发、缓存、回收、成本

03 · 稳定

长时运行、可恢复、可审计

04 · 可扩展

换问题，不换系统

> 模型是 CPU；**定理爆破靠整台机器。**

出错概率：长链路会把 1% 放大

证明图被高频增删改查；单步错误会沿依赖链累积。

端到端成功率 · 示意计算

单步	100 步	1,000 步	10,000 步
99%	37%	0.0043%	≈0
99.9%	90%	37%	0.0045%
99% + 3 次重试	≈100%	99.9%	99%

假设失败独立；真实系统有相关性，重试不是免费的。

错误可检测

Lean、测试、断言先发现

失败可重试

退避、上限、不重复副作用

状态可回滚

检查点、幂等、可恢复

> 系统不是让每一步不犯错；**是让错误不致命**。

系统工程难在哪：六个界面

任何一层单独可用，都不等于整机可靠。

01 · 状态一致性
并发读写同一张图
版本、锁、冲突合并

02 · 调度与预算
先跑谁、跑多久
并发、抢占、成本回收

03 · 失败与重试
错误不重复副作用
幂等、退避、上限、检查点

04 · 验证接口
自然语言 ↔ Lean
语义偏差与假设审查

05 · 可观测性
失败可归因
日志、回放、审计

06 · 人机边界
何时停、谁负责
自动执行，人工把关

低垂的果实：难证明、易证伪；找到一个反例就能定论 · **真正的系统问题**：可复现、可扩展、可维护

低垂的果实证明方向；**真正的难题是整机**。

从下一个 token 到下一个定理

token → 概率 → 候选 → 验证 → 定理

概率负责广度

验证负责正确

人负责方向

> 下一个定理，仍然从**下一个 token** 开始。